

Fachinformatiker für Anwendungsentwicklung
Dokumentation zur schulischen Projektarbeit

Entwicklung von DataPhoenix

Terminalbasiertes Tool zur Wiederherstellung von Daten

Abgabetermin: Twistringen, den 31.05.2026

Auszubildender:

Bastian Rentzsch
Zitadelle 1
27239 Twistringen



Ausbildungsbetrieb:

cbm GbmH
Wegesende 3-4
28195 Bremen

Inhaltsverzeichnis

Abbildungsverzeichnis	III
Tabellenverzeichnis	IV
Listings	V
Abkürzungsverzeichnis	VI
1 Einleitung	1
1.1 Projektumfeld	1
1.2 Projektziel	1
1.3 Projektbegründung	1
1.4 Projektschnittstellen	1
1.5 Projektabgrenzung	1
2 Projektplanung	2
2.1 Projektphasen	2
2.2 Ressourcenplanung	2
3 Analysephase	3
3.1 Ist-Analyse	3
3.2 Wirtschaftlichkeitsanalyse	3
3.2.1 „Make or Buy“-Entscheidung	4
3.2.2 Projektkosten	4
3.2.3 Amortisation	4
3.3 Nutzwertanalyse	4
3.4 Soll-Analyse	5
3.5 Anwendungsfälle	5
4 Entwurfsphase	6
4.1 Zielplattform	6
4.1.1 Programmiersprache: Java	6
4.1.2 Datenbank: Keine (Dateibasierte Speicherung)	6
4.1.3 Client/Server-Architektur: Standalone Desktop-Anwendung	7
4.2 Architekturdesign	7
4.3 Entwurf der Benutzeroberfläche	8
4.4 Geschäftslogik	8
5 Implementierungsphase	9
5.1 Implementierung der Kernkomponenten	9
5.1.1 Master Boot Record Parsing	9
5.1.2 Boot Sector und FAT32 Parameter	10

5.1.3	Directory Entry Parsing	10
5.1.4	Cluster Reading und FAT Navigation	10
5.1.5	Datei-Wiederherstellung	10
5.1.6	Rekursive Verzeichnis-Wiederherstellung	11
5.2	Fehlerbehandlung	11
5.2.1	IOException Behandlung	11
5.2.2	Validierung und Defensive Programmierung	11
5.2.3	Resource Management	12
5.3	Herausforderungen und Lösungen	12
5.3.1	Little-Endian Byte-Parsing	12
5.3.2	Rekursive Directory-Traversierung	12
5.3.3	Memory-Management bei großen Dateien	12
5.3.4	Read-Only forensische Integrität	13
6	Dokumentation	13
6.1	Projektdokumentation	13
6.2	Code-Dokumentation	13
	Literaturverzeichnis	14
A	Anhang	i
A.1	Detaillierte Zeitplanung	i
A.2	Use Case-Diagramm	ii
A.3	Komponentendiagramm	iii
A.4	Klassendiagramm	iv
A.5	Entwicklerdokumentation	v
A.6	Klasse: DirectoryParser	vii

Abbildungsverzeichnis

1	Use Case-Diagramm	ii
2	Komponentendiagramm	iii
3	Klassendiagramm	iv

Tabellenverzeichnis

1	Grobe Zeitplanung	2
2	Wirtschaftlichkeitsanalyse	3
3	Kostenaufstellung	4
4	Nutzwertanalyse	4



Listings

1	Klasse: DirectoryParser	vii
---	-----------------------------------	-----

Abkürzungsverzeichnis

REST	Representational State Transfer
API	Application Programming Interface
BIOS	Basic Input/Output System
BPB	BIOS Parameter Block
FAT	File Allocation Table
FAT16	File Allocation Table 16-bit
FAT32	File Allocation Table 32-bit
NTFS	New Technology File System
GUI	Graphical User Interface
JVM	Java Virtual Machine
JMX	Java Management Extensions
CLI	Command Line Interface
MBR	Master Boot Record
LBA	Logical Block Addressing
LE	Little-Endian

1 Einleitung

1.1 Projektumfeld

Auftraggeber und Kunde des Projekts bin ich selber.

1.2 Projektziel

Ziel des Projektes soll das Ermöglichen von Datenwiederherstellung auf Laufwerken, die mit dem FAT32-Format formatiert sind, sein.

1.3 Projektbegründung

Die Motivation hinter dem Projekt ist zu verstehen, wie ein FAT32-System Daten speichert, löscht und wiederherstellen lässt.

1.4 Projektschnittstellen

Die Anwendung interagiert mit Image-Datei von Datenträgern. Wiederhergestellte Dateien und Ordner werden auf den Computer gespeichert.

1.5 Projektabgrenzung

Das Projekt beschränkt sich auf die Wiederherstellung von Daten auf Datenträgern mit dem FAT32-Format und kein anderes Format wie zum Beispiel FAT16, NTFS usw. Folgende Punkte sind ausdrücklich nicht Bestandteil des Projekts:

- GUI
- Anbindung an externe Datenbanksysteme (PostgreSQL, MySQL)
- Überwachung externer JVM-Prozesse via JMX
- Netzwerkfähigkeit oder REST-API

2 Projektplanung

2.1 Projektphasen

Projektphase	Geplante Zeit
Analysephase	8 h
Entwurfsphase	25 h
Implementierungsphase	35 h
Erstellen der Dokumentation	12 h
Gesamt	80 h

Tabelle 1: Grobe Zeitplanung

Eine detailliertere Zeitplanung befindet sich im Anhang A.1: Detaillierte Zeitplanung auf Seite i.

2.2 Ressourcenplanung

Hardware

- Arbeitsplatz mit Thin-Client

Software

- IntelliJ IDEA 2026.1 – Entwicklungsumgebung
- Oracle OpenJDK 26 - Java Development Kit
- Windows 11 Pro – Betriebssystem
- Github – Versionsverwaltung
- diagrams.net – Programm zur Erstellung von Diagrammen und Modellen
- TeXstudio – Entwicklungsumgebung von $\text{\LaTeX}$
- MiKTeX – Distribution des Textsatzsystems $\text{\LaTeX}$

Personal

- Auszubildender Fachinformatiker für Anwendungsentwicklung – Umsetzung des Projektes

3 Analysephase

3.1 Ist-Analyse

Im aktuellen Zustand bestehen keine praktischen oder theoretischen Kenntnisse über die interne Funktionsweise des FAT32-Dateisystems. Insbesondere fehlt ein Verständnis darüber, wie Daten auf dem Datenträger organisiert werden, wie Dateieinträge strukturiert sind und wie Löschvorgänge im System umgesetzt werden.

Bestehende Programme von Drittanbietern sind zwar verfügbar, werden jedoch im Rahmen dieses Projekts bewusst nicht eingesetzt, da der Fokus auf dem eigenständigen Verständnis und der Entwicklung einer eigenen Lösung liegt.

Ziel ist es, ein terminalbasiertes Tool zu entwickeln, das in der Lage ist, Image-Dateien von FAT32-Datenträgern auszuwerten und gelöschte Daten zumindest teilweise wiederherzustellen. Damit soll nicht nur eine funktionale Anwendung entstehen, sondern auch ein tiefgehendes Verständnis der zugrunde liegenden Speicherstrukturen erreicht werden.

3.2 Wirtschaftlichkeitsanalyse

Bereich	Aufwand	Nutzen
Implementierung	Entwicklung eines terminalbasier-ten Tools	Eigenständige Lösung zur Wiederherstellung gelöschter Daten
Testen	Prüfung mit Image-Dateien und Fehleranalyse	Nachweis der Funktionsfähigkeit
Dokumentation	Erstellung der Projektdokumenta-tion	Nachvollziehbare Darstellung des Projekts
Präsentation	Vorbereitung und Vorstellung vor der Klasse	Vermittlung der Ergebnisse und des Fachwissens
Bewertung	Inhalt	
Aufwand	Insgesamt moderat, da keine zusätzlichen Lizenz- oder Hardwarekosten anfallen.	
Nutzen	Hoch, da ein klar definierter fachlicher und praktischer Mehrwert ent-steht.	
Wirtschaftlichkeit	Das Projekt ist im Verhältnis von Aufwand zu Nutzen wirtschaftlich sinn-voll.	

Tabelle 2: Wirtschaftlichkeitsanalyse

3.2.1 „Make or Buy“-Entscheidung

Es gibt kein fertiges Produkt, dass alle Anforderungen des Projekts abdeckt, da eins der Ziele das Verbessern von meinem Verständnis vom FAT32-Format ist. Aus diesem Grund wird dieses Projekt selbst umgesetzt.

3.2.2 Projektkosten

Die Kosten für die Durchführung des Projekts setzen sich sowohl aus Personal-, als auch aus Ressourcenkosten zusammen. Ich rechne mit dem gesetzlichen Stundensatz von 13,90 €. Für die Nutzung von Ressourcen¹ wird ein Stundensatz von 15 € angenommen. Die Durchführungszeit des Projekts beträgt 80 Stunden. Eine Aufstellung der Kosten befindet sich in Tabelle 3 und sie betragen insgesamt 2312,00 €.

Vorgang	Zeit	Kosten pro Stunde	Kosten
Entwicklungskosten	80 h	13,90 € + 15 € = 28,90 €	2312,00 €
			2312,00 €

Tabelle 3: Kostenaufstellung

3.2.3 Amortisation

Es ist schwierig die Wirtschaftlichkeit dieses Projektes zu errechnen, da kein sofortiger Nutzen erkennbar ist. Da es sich um ein schulisches Projekt handelt, werden die im vorherigen Abschnitt errechneten Projektkosten nicht durch einen Kundenauftrag gedeckt. Das Ergebnis dieses Projektes stellt allerdings das erlernte Wissen vom FAT32-Format dar.

3.3 Nutzwertanalyse

Kriterium	Gewichtung (%)	Bewertung (1–10)	Teilnutzwert
Fachlicher Wissenserwerb (FAT32-Verständnis)	40	10	4,0
Praktische Umsetzung (Tool-Entwicklung)	25	9	2,25
Dokumentation und Nachvollziehbarkeit	25	9	2,25
Kosten-Nutzen-Verhältnis	5	8	0,4
Technische Machbarkeit	5	9	0,45
Gesamtnutzwert	100%	–	9,35

Tabelle 4: Nutzwertanalyse

¹Räumlichkeiten, Arbeitsplatzrechner etc.

3.4 Soll-Analyse

Im Soll-Zustand soll ein funktionales, terminalbasiertes Tool vorliegen, das in der Lage ist, Image-Dateien von FAT32-formatierten Datenträgern zu analysieren und gelöschte Daten wiederherzustellen. Das Programm soll grundlegende Strukturen des FAT32-Dateisystems interpretieren können, insbesondere die File Allocation Table, Verzeichniseinträge sowie die Cluster-Verwaltung.

Darüber hinaus soll ein fundiertes Verständnis darüber erreicht werden, wie FAT32 Daten speichert, wie Löschvorgänge technisch umgesetzt werden und unter welchen Bedingungen eine Wiederherstellung möglich ist. Das Tool muss dabei nicht alle Spezialfälle abdecken, soll jedoch typische Szenarien der Datenwiederherstellung nachvollziehbar und reproduzierbar unterstützen.

Das Ergebnis des Projekts umfasst neben der Software auch eine nachvollziehbare Dokumentation der Funktionsweise sowie eine Präsentation, in der sowohl die technischen Hintergründe als auch die praktische Umsetzung erläutert werden.

3.5 Anwendungsfälle

Während der Analysephase wurde ein Use-Case-Diagramm erstellt, um eine Übersicht über die fachlichen Anwendungsfälle zu erhalten. Dieses Use-Case-Diagramm bildet alle wichtigen Funktionen ab, die für den späteren Benutzer wichtig sind. Es kann im Anhang A.2: Use Case-Diagramm auf Seite ii genauer betrachtet werden. Mithilfe des Diagramms konnten einzelne Features voneinander abgegrenzt werden, um diese dann in der Implementierungsphase zu realisieren.

4 Entwurfsphase

Alle Informationen zum FAT32-Format wurden im Dokument Microsoft FAT Specification² nachgelesen.

4.1 Zielplattform

4.1.1 Programmiersprache: Java

Für die Entwicklung des Projekt wurde Java als Programmiersprache gewählt. Die Entscheidung basiert auf mehreren aussagekräftigen Kriterien:

- **Plattformunabhängigkeit** ist ein zentrales Auswahlkriterium. Java läuft auf jedem Betriebssystem (Windows, Linux, macOS).
- **Binäre Dateizugriffe** werden durch Java hervorragend unterstützt. Die Klassen `java.io.RandomAccessFile` und `java.nio.file.*` bieten präzise Kontrolle über Byte-Operationen, was für die direkte Lesezugriffe auf Disk-Images essentiell ist. Das Tool muss exakt auf bestimmte Byte-Positionen im Dateisystem zugreifen können, um MBR, Boot Sectors und Directory Entries zu lesen.
- **Starke Typisierung** verhindert typische Programmfehler wie Memory-Leaks und Buffer-Overflows. Bei der Arbeit mit kritischen Dateisystem-Strukturen ist Datenintegrität prioritär. Die statische Typisierung von Java hilft, Fehler bereits zur Compile-Zeit zu erkennen.
- **Keine externen Abhängigkeiten** ist ein weiterer Vorteil. Alle benötigten Funktionen sind in der Java Standard Library verfügbar. Das Tool benötigt keine zusätzlichen Bibliotheken, was Deployment und Installation vereinfacht.
- **Ausgezeichnete Debugging-Fähigkeiten** durch IDE-Unterstützung (IntelliJ IDEA, Eclipse) erleichtern die Entwicklung komplexer Byte-Operationen und das Debuggen von Datenstrukturen.

4.1.2 Datenbank: Keine (Dateibasierte Speicherung)

Für dieses Projekt wurde bewusst auf eine Datenbank verzichtet. Das Tool liest ausschließlich Disk-Images und schreibt wiederhergestellte Dateien direkt auf das lokale Dateisystem. Eine Datenbank-Persistenz wäre kontraproduktiv:

- **Keine Persistenz erforderlich:** Metadaten werden nur während der Laufzeit gespeichert
- **Performance-Vorteil:** Direkte Dateizugriffe sind schneller als Datenbank-Abfragen

²Vgl. MICROSOFT CORPORATION [2005]

- **Keine zusätzliche Infrastruktur:** Das Tool ist sofort einsatzbereit ohne Datenbank-Setup
- **Einfachheit:** Weniger Angriffspunkte und einfachere Wartung

4.1.3 Client/Server-Architektur: Standalone Desktop-Anwendung

Das Projekt wurde als **Single-User Desktop-Anwendung** mit konsolenbasierter CLI konzipiert.

Offline-Fähigkeit ist entscheidend für forensische Anwendungen. Das Tool arbeitet vollständig lokal ohne Netzwerkabhängigkeit. Dies ermöglicht den Einsatz in abgeschotteten Umgebungen und Sicherheits-sensitive Szenarien, wo keine Netzwerkverbindungen zulässig sind.

Konsolenbasierte Schnittstelle bietet mehrere Vorteile:

- Einfaches Deployment ohne Installations-Prozess
- Scripting-Fähigkeit für automatisierte Wiederherstellungs-Prozesse
- Geringe Ressourcen-Anforderungen
- Keine grafischen Abhängigkeiten oder Treiber-Probleme

4.2 Architekturdesign

Gewählte Anwendungsarchitektur: Schichtenarchitektur (Layered Architecture)

Für das Projekt wurde eine Schichtenarchitektur gewählt. Diese Architektur unterteilt das System in klar definierte Ebenen, wobei jede Schicht spezifische Verantwortlichkeiten hat und nur mit benachbarten Schichten kommuniziert. Das System besteht aus vier Schichten von oben nach unten:

Die **Präsentationsschicht** bildet die oberste Ebene und verwaltet alle Benutzerinteraktionen. Sie stellt das Menü-System mit sieben Optionen bereit, zeigt farbige Console-Ausgaben und validiert Benutzereingaben.

Die **Business Logik Schicht** bildet das Herzstück des Tools. Sie koordiniert den gesamten Wiederherstellungsprozess, verwaltet die Liste aller erkannten Partitionen, implementiert die rekursive Datei- und Verzeichnis-Wiederherstellung und trackt bereits wiederhergestellte Dateien zur Vermeidung von Duplikaten.

Die **Dateisystem-Abstraktion Schicht** stellt die Dateisystem-Strukturen bereit. Sie enthält den MBR-Parser für die Master Boot Record Struktur, den Partition-Manager für Partitionen und Boot Sectors, den FAT32-Parser für den Boot Sector und BPB, den Directory-Parser für

rekursives Lesen von Directory-Strukturen, und den Cluster-Manager für Cluster-Daten und FAT-Tabelle Navigation.

Die **Infrastruktur-Schicht** bietet grundlegende Funktionen. Sie enthält den Byte-Parser für Little-Endian Konvertierung und den File I/O für read-only Zugriff auf Disk Image Dateien.

Ein genaues Abbild der Schichtenarchitektur befindet sich im Anhang A.3: Komponentendiagramm auf Seite iii.

4.3 Entwurf der Benutzeroberfläche

Für das Projekt wurde eine CLI gewählt statt einer grafischen Oberfläche. Diese Entscheidung ermöglicht einfaches Deployment ohne Installation, minimale Ressourcen-Anforderungen ohne Grafikkarte, vollständige Scripting-Fähigkeit für Automatisierung, schnelle Lernkurve mit nur sieben Menüoptionen, und funktioniert in isolierten forensischen Umgebungen ohne Netzwerk.

Das Menü-System zeigt nach dem Öffnen eines Disk Images sieben nummerierte Optionen an: Change Image, Show Boot Sector Information, Show all Files and Directories, Show all deleted Files and Directories, Show all Files and Directories from a specific Directory, Restore deleted File or Directory, und Quit. Dateien werden in Chunks von 25 Einträgen angezeigt mit einer Pausen-Frage bei längerer Ausgabe. Gelöschte Dateien werden mit einem markanten [DELETED]-Tag in roter Farbe auf gelbem Hintergrund gekennzeichnet. ANSI Escape Codes sorgen für farbige Überschriften: Rot für Hauptmenü, Blau für Sektionen, Hellblau für Suchergebnisse.

Usability-Prinzipien umfassen Konsistenz im Menü-Format, Fehlervermeidung durch Eingabe-Validierung (nur 1-7, keine leeren Namen), Benutzerkontrolle durch Pausen bei langen Ausgaben, sofortige Rückmeldung für jede Aktion mit Erfolgsmeldungen und Fehler-Details, Einfachheit durch sieben essentielle Optionen, und forensische Sicherheit durch Warnungen bei existierenden Dateien.

4.4 Geschäftslogik

Die Geschäftslogik des Projekt konzentriert sich auf vier Kernbereiche: Disk Image Öffnung und Partitionserkennung, Dateisystem-Parsing (MBR, Boot Sector, Directory), Gelöschte Dateien erkennen und anzeigen, sowie Datei- und Verzeichnis-Wiederherstellung. Diese Bereiche werden durch die folgenden Hauptklassen implementiert.

Fat32Reader ist die zentrale Klasse der Business Logik und koordiniert den gesamten Wiederherstellungsprozess. Sie verwaltet die Disk-Verbindung, die Liste aller Partitionen und trackt bereits wiederhergestellte Dateien. Die `open()`-Methode öffnet das Image im Read-Only-Modus und parsen den MBR. `getAllDeletedFilesAndDirectories()` sammelt alle gelöschten Entries. `recoverAllDeletedFileOrDirectory()` implementiert die rekursive Wiederherstellung.

MasterBootRecord parsen die MBR-Struktur am Offset 446 und verwaltet vier **PartitionEntry**-Objekte mit Start-LBA, Größe und Partitionstyp. **Partition** repräsentiert eine logische **Partition** mit Byte-Grenzen, hält das **BootSector**-Objekt und sammelt alle **DirectoryEntry**-Objekte. **BootSector** parsen den FAT32 BIOS Parameter Block und berechnet Cluster-Count, Cluster-Size aus den BPB-Offsets.

DirectoryParser traversiert Directory-Ketten über Cluster-Chains rekursiv und rekonstruiert gelöschte Dateien durch sequentielles Cluster-Lesen. **ClusterReader** berechnet Byte-Offsets für Cluster, liest FAT-Einträge für Cluster-Navigation und parsen 32-Byte Directory Entries inklusive UTF-16 LE Long Filename Decodierung. **DirectoryEntry** repräsentiert Dateien/Verzeichnisse mit Filename, Start-Cluster, Größe und Deleted-Flag (0xE5).

LittleEndianParser bietet Byte-Parsing für Little-Endian Konvertierung und UTF-16 Decodierung. Der Wiederherstellungs-Flow beginnt mit `open()`, gefolgt von `getAllDeletedFilesAndDirectories()` oder `recoverAllDeletedFileOrDirectory()`, wobei rekursiv `readDirectory()` und `readCluster()` aufgerufen werden bis Dateidaten in `byte[]` rekonstruiert und mit `Files.write()` gespeichert sind.

Ein genaues Abbild des Klassendiagramms befindet sich im Anhang A.4: Klassendiagramm auf Seite iv.

5 Implementierungsphase

5.1 Implementierung der Kernkomponenten

5.1.1 Master Boot Record Parsing

Die **MasterBootRecord**-Klasse parsen die MBR-Struktur am Offset 446 im ersten Sektor. Der Konstruktor liest 16 Bytes pro Partitionseintrag für vier Einträge. Jeder Eintrag wird mit `ByteBuffer.wrap()` umhüllt und `ByteOrder.LITTLE_ENDIAN` gesetzt da x86 Little-Endian verwendet.

Die **PartitionEntry**-Klasse extrahiert sechs Felder: `flag` (Offset 0), `startCHS` (Offset 1), `type` (Offset 4), `endCHS` (Offset 5), `startLBA` (Offset 8) und `size` (Offset 12). Alle Multi-Byte-Werte werden mit `getInt()` oder `get()` gelesen und mit Bit-Maskierung in unsigned Werte umgewandelt.

5.1.2 Boot Sector und FAT32 Parameter

Die BootSector-Klasse parsen den 512-Byte BPB. Wichtige Felder werden an festen Positionen gelesen: bytesPerSector (Offset 11), sectorsPerCluster (Offset 13), reservedSectorsCount (Offset 14), fatsCount (Offset 16), totalSectors (Offset 32), fatSize (Offset 36) und rootCluster (Offset 44).

Abgeleitete Parameter werden berechnet:

$$dataSector = totalSectors - reservedSectorsCount - (fatsCount * fatSize)$$

$$clusterCount = dataSector / sectorsPerCluster$$

$$clusterSize = bytesPerSector * sectorsPerCluster$$

$$firstDataSector = reservedSectorsCount + (fatsCount * fatSize)$$

5.1.3 Directory Entry Parsing

Die ClusterReader-Klasse parsen 32-Byte Directory Entries. Das erste Byte zeigt Status: 0x00 = unused (Ende), 0xE5 = deleted, anderer Wert = existierend. Das Attribute-Byte bei Offset 11 bestimmt den Typ: Bit 0x10 = Verzeichnis, Bit 0x0F = Long Filename Entry.

Long Filename Entries enthalten UTF-16 LE kodierten Text über Offset 1-10, 14-26 und 28-30. Die readUTF16LEChars()-Methode liest in 2-Byte Schritten und kombiniert Parts in reverse Reihenfolge. Start-Cluster wird aus high (Offset 20) und low (Offset 26) kombiniert: (high « 16) | low. fileSize wird bei Offset 28 gelesen.

5.1.4 Cluster Reading und FAT Navigation

Die readCluster()-Methode berechnet Byte-Offset:

$$offset = startByte + ((cluster - 2) * sectorsPerCluster + firstDataSector) * bytesPerSector$$

Cluster-Nummern beginnen bei 2 (Cluster 0 und 1 sind reserviert).

Die getNextCluster()-Methode liest FAT-Eintrag bei fatStartByte + (cluster * 4). Vier Bytes werden zu 32-bit kombiniert und mit 0xFFFFFFFF maskiert (FAT32 verwendet nur lower 28 Bits). Werte ab 0xFFFFFFFF8 sind End-of-Chain-Marker.

5.1.5 Datei-Wiederherstellung

Die recoverFile()-Methode rekonstruiert gelöschte Dateien.

$$expectedClusters = (fileSize + clusterSize - 1) / clusterSize$$

Sequenziell wird durch Cluster-Nummern iteriert mit readCluster() für jeden.

Jeder Cluster wird auf Leerheit geprüft (alle Bytes = 0). Leere Cluster nach dem ersten bedeuten Datei-Ende. dataChunks werden zu byte[] result zusammengefügt. Die Größe ist min(fileSize, totalSize) aus dataChunks um Buffer-Overflow zu vermeiden.

5.1.6 Rekursive Verzeichnis-Wiederherstellung

Die recoverAllDeletedFileOrDirectory()-Methode implementiert rekursive Wiederherstellung. Zuerst wird recoveredFiles.contains(name) geprüft um Duplikate zu vermeiden. Dann wird Files.createDirectories() für Zielverzeichnis aufgerufen.

Für jede Partition wird durch directoryEntries iteriert. Bei match fileName und isDeleted=true: bei Verzeichnissen wird readDirectory() für Sub-Entries aufgerufen und rekursiv recoverAllDeletedFileOrDirectory() für jedes Sub-Entry. Bei Dateien wird recoverFile() für Daten-Rekonstruktion aufgerufen und Files.write() zum Speichern. Datei wird nur geschrieben wenn nicht existent (Files.exists()-Prüfung).

Die Klasse `DirectoryParser` befindet sich im Anhang A.6: Klasse: `DirectoryParser` auf Seite vii zum genaueren ansehen von einem Code-Beispiel.

5.2 Fehlerbehandlung

5.2.1 IOException Behandlung

Alle Methoden die Dateizugriffe durchführen werfen IOException wenn Fehler auftreten. Die open()-Methode wirft IOException bei ungültigem Pfad oder wenn die Datei nicht existiert. Die recoverFile()-Methode wirft IOException bei Lesefehlern während des Cluster-Parsings.

In der Main-Klasse wird IOException in zwei catch-Blöcken gefangen: FileNotFoundException für nicht gefundene Images und allgemeine IOException für andere Lesefehler. Beide geben benutzerfreundliche Meldungen aus mit Class-Name und Message für Debugging.

5.2.2 Validierung und Defensive Programmierung

Die open()-Methode validiert den imagePath auf null oder empty und wirft IOException bei ungültiger Eingabe. Die recoverFile()-Methode prüft ob startCluster ≥ 2 um invalid Cluster zu vermeiden. Die getNextCluster()-Methode prüft ob offset + 4 > disk.length() um Out-of-Bounds-Zugriffe zu vermeiden.

Das recoveredFiles-Set verhindert Endlosschleifen bei rekursiver Verzeichnis-Wiederherstellung. Das visited-Set in DirectoryParser verhindert Endlosschleifen bei korrupten Cluster-Chains die Zyklen enthalten könnten.

5.2.3 Resource Management

Die `RandomAccessFile` wird in `Fat32Reader` als Instanzvariable gehalten und bleibt während der gesamten Lebensdauer des Objekts geöffnet. Der Benutzer kann das Image mit Option 1 wechseln was eine neue `open()`-Ausführung mit neuem `RandomAccessFile`-Objekt auslöst. Das alte `RandomAccessFile`-Objekt wird vom Garbage Collector freigegeben wenn keine Referenzen mehr existieren.

Der Scanner wird am Ende des Programms mit `scanner.close()` geschlossen. Alle Byte-Arrays werden als lokale Variablen in Methoden gehalten und automatisch freigegeben nach Method Ende.

5.3 Herausforderungen und Lösungen

5.3.1 Little-Endian Byte-Parsing

Die Haupt-Herausforderung war korrektes Little-Endian Parsing von FAT32-Strukturen. Java verwendet standardmäßig Big-Endian für `ByteBuffer` was zu falschen Werten führte. Die Lösung war explizites Setzen von `ByteOrder.LITTLE_ENDIAN` auf allen `ByteBuffer`-Instanzen und manuelles Bit-Shifting in `LittleEndianParser` für maximale Kontrolle. Long Filename UTF-16 Decodierung

Long Filename Entries verwenden UTF-16 little-endian Kodierung die nicht nativ für Java ist. Die Herausforderung war korrekte Decodierung von multi-Byte Zeichen über mehrere Entry-Parts. Die Lösung war `readUTF16LEChars()` die in 2-Byte Schritten liest und Zeichenparts in reverse Reihenfolge kombiniert, da Parts von hinten nach vorne sortiert sind.

5.3.2 Rekursive Directory-Traversierung

Rekursive Verzeichnis-Parsing führte zu Endlosschleifen bei korrupten Images mit zyklischen Cluster-Chains. Die Lösung war ein `visited-Set<Long>` das alle bereits besuchten Cluster-Nummern trackt und doppelte Cluster überspringt.

5.3.3 Memory-Management bei großen Dateien

Wiederherstellung sehr großer Dateien (> 1 GB) führte zu `OutOfMemoryError` wenn komplette Datei in Memory geladen wurde. Die Lösung war chunkweise Verarbeitung wobei `expectedClusters` berechnet wird und nur einzelne Cluster gleichzeitig gelesen werden. `dataChunks` werden in einer `List<byte[]>` gespeichert und erst am Ende zu einem `byte[]` zusammengefügt mit der exakten `fileSize`.

5.3.4 Read-Only forensische Integrität

Die Anforderung dass Original-Daten niemals verändert werden müssen war kritisch. Die Lösung war `RandomAccessFile` immer im Modus `read-only` zu öffnen. Jede `write()`-Operation wird auf die Ziel-Datei beschränkt nie auf die Disk-Image-Datei. Dies garantiert forensische Integrität des Original Images.

6 Dokumentation

6.1 Projektdokumentation

Diese Projektdokumentation wurde mit der $\text{\LaTeX}$ -Vorlage wurde von Stefan Macke³ erstellt.

6.2 Code-Dokumentation

Alle Klassen erhalten JavaDocs-Kommentare am Anfang der Klassendeklaration beschreibend die Verantwortung der Klasse. Alle öffentlichen Methoden erhalten JavaDocs mit `@param` und `@return` Tags beschreibend Parameter und Rückgabewerte.

Wichtige Algorithmen erhalten Inline-Kommentare: Die Formel für Cluster-Offset-Berechnung wird kommentiert, die Bit-Maskierung für unsigned Werte wird erklärt, die Long Filename UTF-16 Decodierung wird beschrieben.

Die Code-Dokumentation wurde mittels Javadoc automatisch generiert. Ein beispielhafter Auszug aus der Dokumentation einer Klasse findet sich im Anhang A.5: Entwicklerdokumentation auf Seite v.

³Blog des Autors:<http://fachinformatiker-anwendungsentwicklung.net>, Twitter:@StefanMacke

Literaturverzeichnis

Microsoft Corporation 2005

MICROSOFT CORPORATION: *Microsoft FAT Specification*. https://academy.cba.mit.edu/classes/networking_communications/SD/FAT.pdf. Version: August 2005

A Anhang

A.1 Detaillierte Zeitplanung

Analysephase	8 h
1. Analyse des Ist-Zustands	3 h
2. Wirtschaftlichkeitsanalyse	2 h
3. Erstellen eines Anwendungsfalldiagramm	3 h
Entwurfsphase	25 h
1. Einlesen in das FAT32-Format	18 h
2. Erstellen eines Klassendiagramms	7 h
Implementierungsphase	35 h
1. Auslesen des Bootsektoren der Partitionen vom Image	10 h
2. Lesen des Dateisystem vom Image	13 h
3. Anzeigen von allen gelöschten Dateien/Ordner	1 h
4. Ordnersuche mit Anzeige aller Dateien und Unterordner	3 h
5. Wiederherstellung von Datei/Ordner bei Namen	8 h
Erstellen der Dokumentation	12 h
1. Erstellen der Projektdokumentation	10 h
2. Erstellen der Javadoc	2 h
Gesamt	80 h

A.2 Use Case-Diagramm

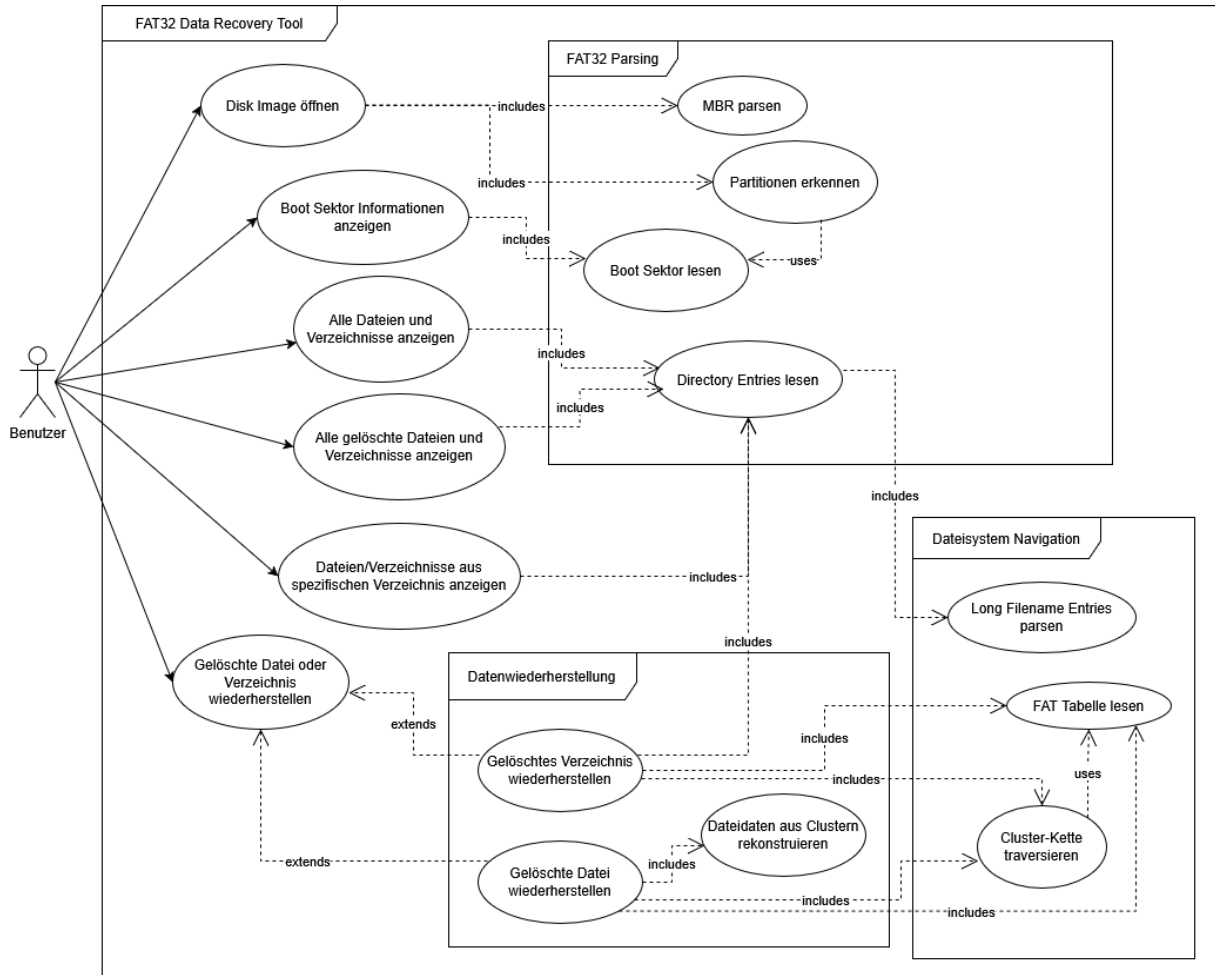


Abbildung 1: Use Case-Diagramm

A.3 Komponentendiagramm

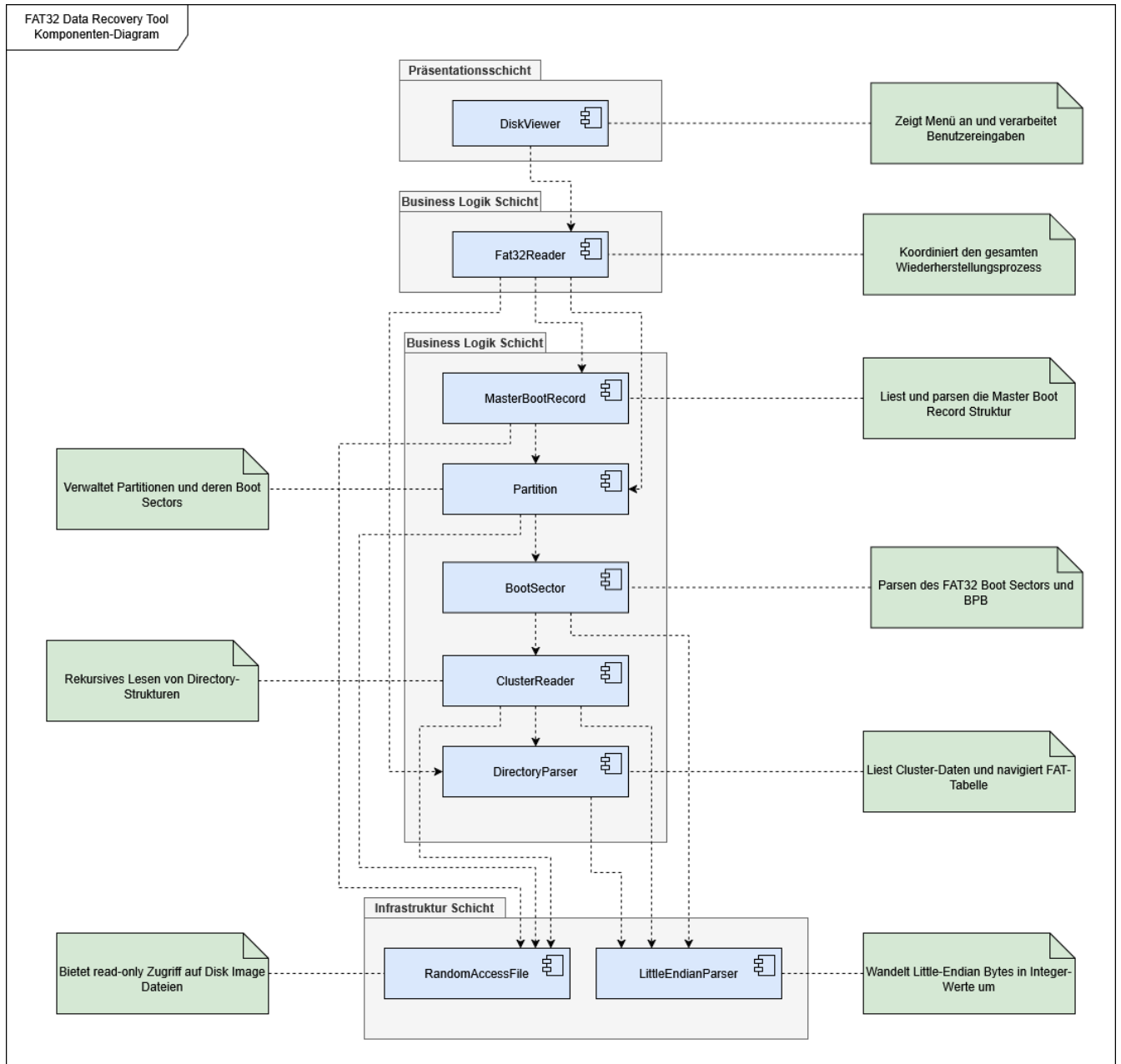


Abbildung 2: Komponentendiagramm

A.4 Klassendiagramm

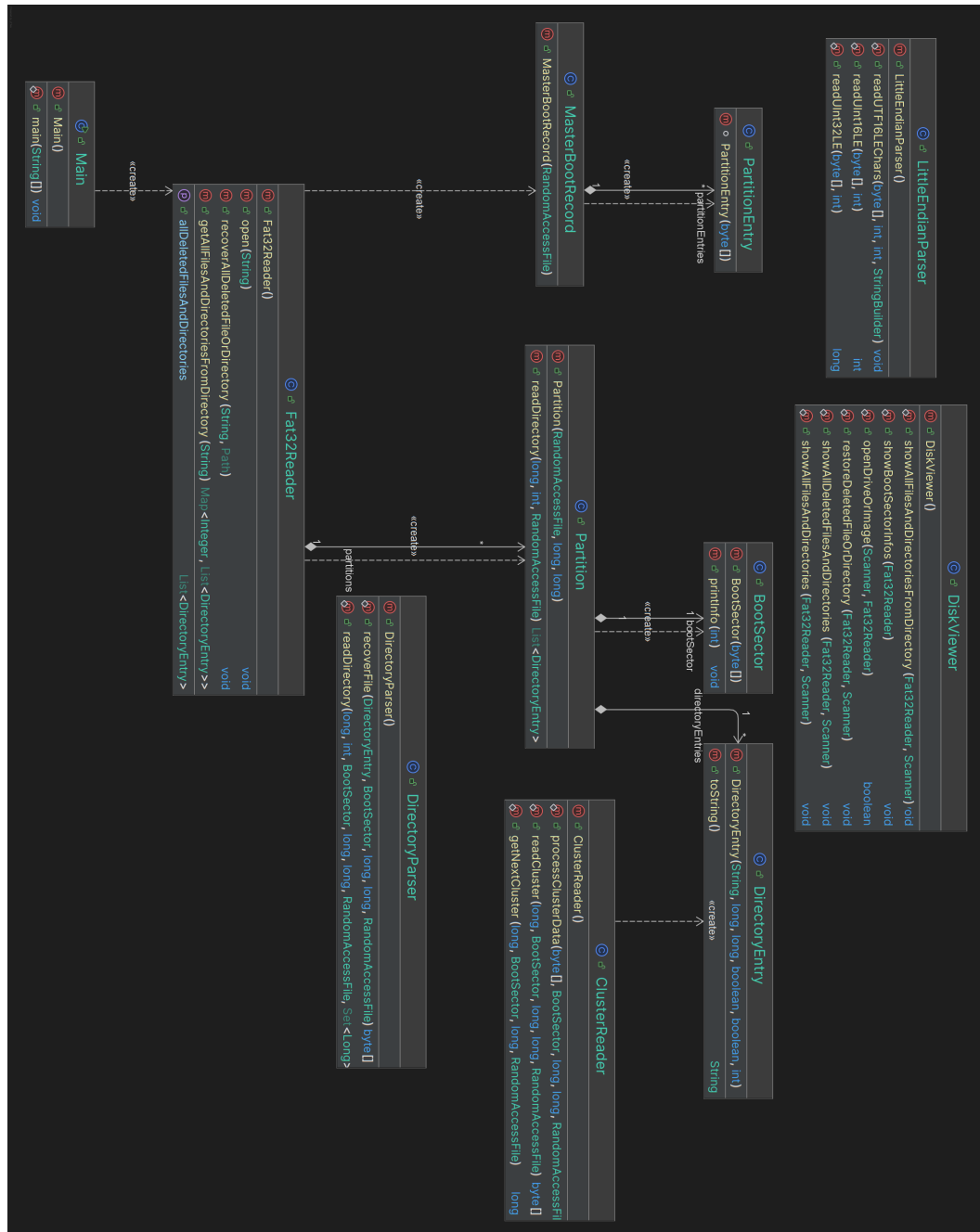


Abbildung 3: Klassendiagramm

A.5 Entwicklerdokumentation

MasterBootRecord

file:///C:/Users/bastianr/Desktop/Data%20Recovery%20Tool/docs/ma...

Klasse MasterBootRecord

[java.lang.Object](#)
masterBootRecord.MasterBootRecord

```
public class MasterBootRecord  
extends Object
```

MasterBootRecord represents the Master Boot Record (MBR) from the first sector (512 bytes) of a disk image.

The MBR contains:

- Bootstrap code (bytes 0-445)
- Four partition entries (bytes 446-509, 16 bytes each)
- Boot signature (bytes 510-511, should be 0x55AA)

This class parses the four partition entries starting at offset 446 and stores them in a PartitionEntry array.

Version:

1.0

Autor:

Bastian Rentzsch

Feldübersicht

Felder		
Modifikator und Typ	Feld	Beschreibung
PartitionEntry[]	partitionEntries	Array of four partition entries (may contain nulls for empty entries)

Konstruktorübersicht

Konstruktoren	
Konstruktor	Beschreibung
MasterBootRecord (RandomAccessFile disk)	Constructs a MasterBootRecord by reading four partition entries from disk.

Methodenübersicht

Von Klasse geerbte Methoden Object
equals , getClass , hashCode , notify , notifyAll , toString , wait , wait

MasterBootRecord

file:///C:/Users/bastianr/Desktop/Data%20Recovery%20Tool/docs/ma...

```
    , wait
```

Felddetails

partitionEntries

```
public PartitionEntry[] partitionEntries
```

Array of four partition entries (may contain nulls for empty entries)

Konstruktordetails

MasterBootRecord

```
public MasterBootRecord(RandomAccessFile disk)
    throws IOException
```

Constructs a MasterBootRecord by reading four partition entries from disk.

Reads 16 bytes for each partition entry starting at offset 446:

- Entry 0: offset 446
- Entry 1: offset 462
- Entry 2: offset 478
- Entry 3: offset 494

Each entry is passed to the PartitionEntry constructor for parsing.

Parameter:

disk - the RandomAccessFile handle for the disk image

Löst aus:

IOException - if reading partition entries fails

A.6 Klasse: DirectoryParser

Kommentare werden nicht angezeigt.

```
1 package filesystem;
2
3 import java.io.IOException;
4 import java.io.RandomAccessFile;
5 import java.util.ArrayList;
6 import java.util.List;
7 import java.util.Set;
8
9
10 public class DirectoryParser {
11
12     public static List<DirectoryEntry> readDirectory(long startCluster,
13                                                     int level,
14                                                     BootSector bootSector,
15                                                     long startByte,
16                                                     long endByte,
17                                                     RandomAccessFile disk,
18                                                     Set<Long> visited) throws IOException {
19         List<DirectoryEntry> entries = new ArrayList<>();
20
21         if (!visited.add(startCluster)) return entries;
22
23         long currentCluster = startCluster;
24
25         while (currentCluster >= 2 && currentCluster < bootSector.clusterCount + 2) {
26             byte[] clusterData = ClusterReader.readCluster(currentCluster, bootSector, startByte, endByte,
27                                                             disk);
28
29             ClusterReader.processClusterData(clusterData, bootSector, startByte, endByte, disk, entries,
30                                             level, visited);
31
32             long nextCluster = ClusterReader.getNextCluster(currentCluster, bootSector, startByte, disk);
33
34             if (nextCluster < 2 || nextCluster >= bootSector.clusterCount + 2) break;
35
36             currentCluster = nextCluster;
37         }
38
39         return entries;
40     }
41
42     public static byte[] recoverFile(DirectoryEntry directoryEntry,
43                                     BootSector bootSector,
44                                     long startByte,
45                                     long endByte,
46                                     RandomAccessFile disk) throws IOException {
47         if (directoryEntry.isDirectory || directoryEntry.startCluster < 2) return new byte[0];
48     }
49 }
```

```
46
47     List<byte[]> dataChunks = new ArrayList<>();
48     long currentCluster = directoryEntry.startCluster;
49
50     int expectedClusters = (int) ((directoryEntry.fileSize + bootSector.clusterSize - 1) / bootSector.
51         clusterSize);
52
53     for (int i = 0; i < expectedClusters; i++) {
54         byte[] clusterData = ClusterReader.readCluster(currentCluster, bootSector, startByte, endByte,
55             disk);
56
57         if (clusterData.length == 0) break;
58
59         boolean isEmpty = true;
60         for (byte b : clusterData) {
61             if (b != 0) {
62                 isEmpty = false;
63                 break;
64             }
65         }
66
67         if (isEmpty && i > 0) break;
68
69         dataChunks.add(clusterData);
70
71         currentCluster++;
72     }
73
74     long totalSize = 0;
75     for (byte[] chunk : dataChunks) {
76         totalSize += chunk.length;
77     }
78
79     long fileSize = Math.min(directoryEntry.fileSize, totalSize);
80     byte[] result = new byte[(int) fileSize];
81
82     int offset = 0;
83     for (byte[] chunk : dataChunks) {
84         int copyLength = (int) Math.min(chunk.length, fileSize - offset);
85         if (copyLength <= 0) break;
86         System.arraycopy(chunk, 0, result, offset, copyLength);
87         offset += copyLength;
88     }
89
90     return result;
91 }
```

Listing 1: Klasse: DirectoryParser